

```
int main () {

    mqd_t mq_create, mq_snd_open;
    int mq_ret, prio;
    long int len;
    char buf[8192];

    mq_snd_open=mq_open("/mymq", O_WRONLY);

    printf ("Please enter the message : ");
    scanf ("%[^\\n]", buf);
    len=strlen(buf);
    printf (" Please enter the message priority: ");
    scanf ("%d", &prio);

    mq_ret=mq_send(mq_snd_open, buf, len+1, prio);
    if(mq_ret == 0)
        printf (" message is sent successfully...\\n");
    else
        printf (" Message is not sent ....\\n");

}
```

Figure 7: Message sending in POSIX with priority

```
int main () {
    mqd_t mq_rec_open;
    char buf[8192];
    int ret_rec, prio;

    mq_rec_open=mq_open("/mymq", O_RDONLY);
    ret_rec=mq_receive(mq_rec_open, buf, sizeof(buf), &prio);
    printf ("Prio: %d Message: %s\\n", prio, buf);
}
```

Figure 8: Message receiving in POSIX with priority

print the message received by the process stored in the buffer and the priority of the received message. Figure 8 depicts the way to receive a higher priority message.

Timed sending and receiving

Other variations of the send and receive functions are *mq_timedreceive* and *mq_timedsend*; an addition to these variations is the *timespec* structure. The *timespec* construct is defined, and can be accessed and used by the programmer to specify the time ceiling with nanosecond accuracy. These calls are used when we do not use the *O_NONBLOCK* flag; instead, we specify the time ceiling, which after a countdown blocks messages until there is enough space on the message queue to send them.

In *blocking receive*, the process blocks a message unless the message is made available in the queue, whereas in *non-blocking receive*, the process extracts a null or a message that is available from the queue.

In *blocking send*, the process blocks a message unless the message has been received by the receiving end, and in a *non-blocking send*, the process places the message on the message queue and terminates or performs another operation.

In the sending and receiving process, we have used *mq_timedsend* and *mq_timedreceive* by mentioning the wait time as 5 seconds. In this case, there is a timeout after 5 seconds if the message has not been received or sent.

The sending process opens the message queue in write-only mode. We use the *clock_gettime()* function and pass the *CLOCK_REALTIME* parameter to get the approximate time now.

```
int main () {

    mqd_t mq_create, mq_snd_open;
    int mq_ret;
    long int len;
    char buf[8192];

    struct timespec tv;
    clock_gettime(CLOCK_REALTIME, &tv);
    tv.tv_sec += 5;

    mq_snd_open=mq_open("/mymq", O_WRONLY);

    printf ("Please enter the message : ");
    scanf ("%[^\\n]", buf);
    len=strlen(buf);

    mq_ret=mq_timedsend(mq_snd_open, buf, len+1, 0, &tv);
    if(mq_ret == -1)
        printf ("Message cannot be sent to the mq...\\n");
    else
        printf (" Message is sent successfully...\\n");

}
```

Figure 9: Message sending with *mq_timedsend()*

```
int main () {
    mqd_t mq_rec_open;
    char buf[8192];
    int ret_rec;

    struct timespec tv;
    clock_gettime(CLOCK_REALTIME, &tv);
    tv.tv_sec += 5;

    mq_rec_open=mq_open("/mymq", O_RDONLY);
    ret_rec=mq_timedreceive(mq_rec_open, buf, sizeof(buf), NULL, &tv);

    if (ret_rec == -1)
        printf (" There is no message available in the mq...\\n");
    else
        printf ("Message: %s\\n", buf);

}
```

Figure 10: Message receiving with *mq_timedreceive()*

Figure 9 explains the procedure to send messages to the message queue with timeout. In *mq_timedsend*, the first four parameters remain the same as *mq_send*, and the fifth added parameter is the time the sending process needs to wait for the receiving process to extract the message from the queue. If -1 is returned, we print the message “Message cannot be sent to the mq...”; else, we print the message “Message is sent successfully...” indicating that the message has been received successfully.

We follow a similar procedure for the *mq_timedreceive* system call at the receiving end, and mention the time needed for the process to wait before it needs to terminate without having extracted a message from the queue. Figure 10 shows the code for message receiving with the *mq_timedreceive* system call.

POSIX message queue is suitable for real-time applications since it has the features that can predict the behaviour of the data that communicates between processes. In this article, we have explained the importance of the POSIX message queue, including the procedures to create, send, receive, set priority, and set timeout features for it, along with demo programs. 

By: Mrinalini Raghavendran and B. Thangaraju

Mrinalini Raghavendran is a Linux enthusiast.

B. Thangaraju is associated with the open source technology lab in the International Institute of Information Technology, Bengaluru.